

Week 3: Dynamic Programming & Model-Free Prediction

Mayuresh Thengzode 25B1816

1 Part 1: Dynamic Programming Pseudocode

Dynamic Programming (DP) assumes full structural knowledge of the Markov Decision Process (MDP), explicitly utilizing transition probabilities $P_{ss'}^a$ and the reward function R_s^a .

1.1 Iterative Policy Evaluation

This algorithm evaluates a given policy π by iteratively solving the Bellman expectation equation for $V(s)$.

Algorithm 1: Iterative Policy Evaluation

Input: Policy π to be evaluated

1 $\theta > 0$ (a small threshold determining estimation accuracy) Initialize $V(s)$ arbitrarily (e.g., 0) for all $s \in \mathcal{S}$, and $V(\text{terminal}) = 0$

2 **repeat**

3 $\Delta \leftarrow 0$

4 **for each** $s \in \mathcal{S}$ **do**

5 $v \leftarrow V(s)$

6 $V(s) \leftarrow \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s', r} P(s', r | s, a) [r + \gamma V(s')]$

7 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

8 **until** $\Delta < \theta$

1.2 Policy Iteration

Policy Iteration alternates between fully evaluating the current policy and improving it greedily with respect to the updated value function.

Algorithm 2: Policy Iteration

```
1 1. Initialization
2 Initialize  $V(s) \in \mathbb{R}$  and  $\pi(s) \in \mathcal{A}(s)$  arbitrarily for all  $s \in \mathcal{S}$ 
3 2. Policy Evaluation
4 repeat
5    $\Delta \leftarrow 0$ 
6   for each  $s \in \mathcal{S}$  do
7      $v \leftarrow V(s)$ 
8      $V(s) \leftarrow \sum_{s',r} P(s', r \mid s, \pi(s)) [r + \gamma V(s')]$ 
9      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
10 until  $\Delta < \theta$ 
11 3. Policy Improvement
12  $\text{policy\_stable} \leftarrow \text{true}$ 
13 for each  $s \in \mathcal{S}$  do
14    $\text{old\_action} \leftarrow \pi(s)$ 
15    $\pi(s) \leftarrow \arg \max_a \sum_{s',r} P(s', r \mid s, a) [r + \gamma V(s')]$ 
16   if  $\text{old\_action} \neq \pi(s)$  then
17      $\text{policy\_stable} \leftarrow \text{false}$ 
18 if  $\text{policy\_stable} = \text{true}$  then
19   return  $V$  and  $\pi$ 
20 else
21   Go to Step 2
```

1.3 Value Iteration

Value Iteration continuous evaluation and improvement into a single step by turning the Bellman optimality equation directly into an iterative update rule.

Algorithm 3: Value Iteration

```
1  $\theta > 0$  Initialize  $V(s)$  arbitrarily for all  $s \in \mathcal{S}$ , and  $V(\text{terminal}) = 0$ 
2 repeat
3    $\Delta \leftarrow 0$ 
4   for each  $s \in \mathcal{S}$  do
5      $v \leftarrow V(s)$ 
6      $V(s) \leftarrow \max_a \sum_{s',r} P(s', r \mid s, a) [r + \gamma V(s')]$ 
7      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
8 until  $\Delta < \theta$ 
9 Output a deterministic optimal policy  $\pi \approx \pi^*$  such that:
10  $\pi(s) = \arg \max_a \sum_{s',r} P(s', r \mid s, a) [r + \gamma V(s')]$ 
```

2 Part 2: Toy MDP Comparison — MC vs. TD Learning

We evaluate updates using a simple 3-state deterministic Markov Reward Process (MRP).

2.1 The Setup

$$A \xrightarrow{R=0} B \xrightarrow{R=0} C \xrightarrow{R=+100} \text{Terminal}$$

- Discount factor (γ) = 1.0
- Learning rate (α) = 0.5
- Initial values: $V(A) = 0, V(B) = 0, V(C) = 0$

We process a single observed trajectory:

$$\text{Episode 1: } A \rightarrow (R = 0) \rightarrow B \rightarrow (R = 0) \rightarrow C \rightarrow (R = 100) \rightarrow \text{Terminal}$$

2.2 Monte Carlo (MC) Approach

MC waits for the episode to terminate and updates estimates using the actual downstream total return (G_t).

$$\text{Update rule: } V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

Because the +100 reward occurs at the terminal step, the empirical return from each state in this sequence is identical: $G_A = 100, G_B = 100, G_C = 100$.

$$V(C) \leftarrow 0 + 0.5 \times (100 - 0) = 50$$

$$V(B) \leftarrow 0 + 0.5 \times (100 - 0) = 50$$

$$V(A) \leftarrow 0 + 0.5 \times (100 - 0) = 50$$

Takeaway: In a single episode, reward information propagates completely back to the start state A .

2.3 Temporal Difference Learning — TD(0)

TD(0) updates estimates step-by-step during the episode by bootstrapping from the immediate reward and the current value estimate of the next state.

$$\text{Update rule: } V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

Processing sequentially along the trajectory:

1. **Step 1** ($A \rightarrow B, R = 0$):

$$V(A) \leftarrow 0 + 0.5 \times (0 + V(B) - 0) = 0 + 0.5(0 + 0 - 0) = 0$$

2. **Step 2** ($B \rightarrow C, R = 0$):

$$V(B) \leftarrow 0 + 0.5 \times (0 + V(C) - 0) = 0 + 0.5(0 + 0 - 0) = 0$$

3. **Step 3** ($C \rightarrow \text{Terminal}, R = 100$):

$$V(C) \leftarrow 0 + 0.5 \times (100 + V(\text{Terminal}) - 0) = 0 + 0.5(100 + 0 - 0) = 50$$

Final Values: $V(A) = 0, V(B) = 0, V(C) = 50$

Takeaway: Information only flows backward one step per episode. State C discovers the terminal reward, but state B learns nothing because $V(C)$ was still evaluated as 0 during B 's update window.